

DISEÑO GENERAL

Arquitectura de Microservicios con Docker + Go

VERSIÓN 1.0

KING BUSTAMANTE JENNY

ULATE OBANDO EVELYN VANESSA

Contents

1)	Resumen	3
2)	Arquitectura Lógica y Física	3
2.2)	Arquitectura Física – Infraestructura Docker	5
3)	Descripción de Microservicios y Responsabilidades	6
3.1)	NGINX – LOAD BALANCER	6
3.2)	API Go – R1 y R2	6
3.3)	Keycloak – auth + jwt	6
3.4)	Middleware – jwt + roles	7
3.5)	repository – capa de abstracción de datos	7
3.6)	mongo-router	8
3.7)	search-service	9
4)	Flujo de Datos y LoadBalances	9
4.1	Flujo de Autenticación	9
4.2	Flujo de Request Autenticado (CRUD)	9
4.3	Flujo de Búsqueda	10
5)	Persistencia y Estrategia de BD	10
6)	Caché y Búsqueda	10
6.1)	Redis	10
6.2)	ElasticSearch	10
7)	CI/CD — GitHub Actions	11
8)	Escalabilidad – kubernetes	11

1) RESUMEN

Este documento describe el diseño general del sistema de la plataforma de API construida sobre una arquitectura de microservicios contenerizada en Docker. El sistema provee autenticación federada mediante Keycloak + JWT, balanceo de carga con NGINX, réplicas de API en Go, persistencia dual (MongoDB sharded cluster + PostgreSQL), caché con Redis TTL, búsqueda full-text con Elasticsearch, y un servicio de solicitudes (Resquest) independiente.

La arquitectura fue diseñada para soportar alta disponibilidad, escalabilidad horizontal y cambio transparente entre motores de base de datos mediante la variable de entorno DB_DRIVER (postgres | mongo).

2) ARQUITECTURA LÓGICA Y FÍSICA

El sistema se organiza en cinco capas lógicas claramente delimitadas:

Capa 1 — Entrada

- Usuario externo → NGINX Load Balancer (puerto 80). Todo el tráfico entrante pasa por NGINX que actúa como reverse proxy y punto único de entrada.

Capa 2 — Autenticación

- NGINX redirige peticiones de autenticación a Keycloak. Keycloak emite tokens JWT que son validados por el Middleware en cada request subsiguiente.

Capa 3 — Procesamiento

- API Go (Replica 1 y Replica 2) procesan la lógica de negocio. El Middleware valida JWT y roles antes de permitir el acceso a la capa de datos.

Capa 4 — Datos

- Repository abstrae el acceso a datos. Dependiendo de DB_DRIVER, enruta a PostgreSQL (relacional) o al Mongo-router del cluster sharded de MongoDB.

Capa 5 — Servicios Auxiliares

- Redis (caché TTL), Elasticsearch (búsqueda), Resquest (servicio externo de solicitudes), Search-service (Go, puerto 8080).

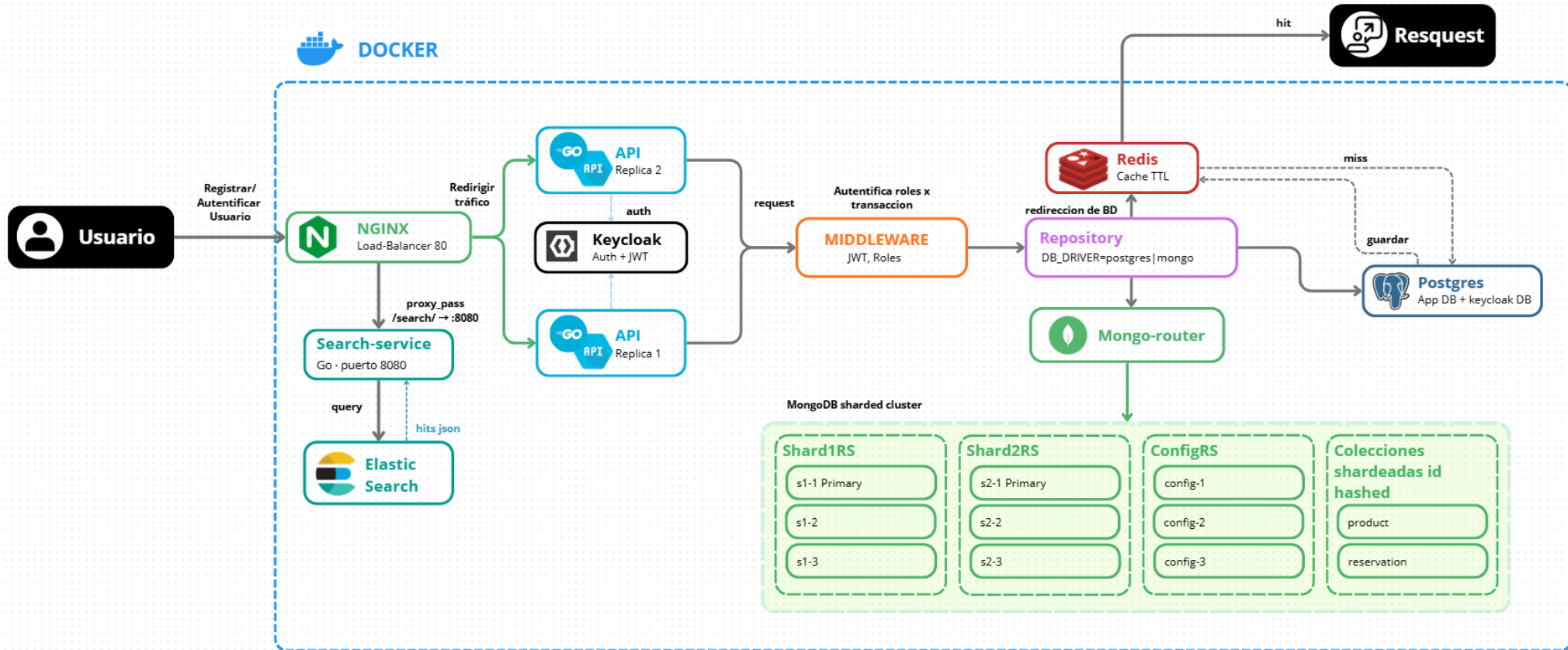


Figura 1: Diagrama de Arquitectura Lógica y de Despliegue

2.2) ARQUITECTURA FÍSICA – INFRAESTRUCTURA DOCKER

Todos los componentes corren como contenedores dentro de una red Docker interna. La siguiente tabla resume los contenedores/servicios y sus puertos:

Tabla 1 – Contenedores Docker del Sistema

Contenedor	Imagen	Puerto	Rol
Nginx	nginx:alpine	80 (host)	Load Balancer / Reverse Proxy
Api-replica-1	golang:1.22-alpine	interno	API Go — instancia primaria
Api-replica-2	golang:1.22-alpine	interno	API Go — instancia secundaria
Keycloak	quay.io/keycloak/keycloak	interno	Auth + emisión JWT
Search-service	golang:1.22-alpine	8080 (host)	Servicio de búsqueda Go
Elasticsearch	elasticsearch:8.x	9200 interno	Motor de búsqueda full-text
Redis	redis:7-alpine	6379 interno	Caché TTL
Mongo-router	mongo:7	27017 interno	Enrutador del cluster
Shard1RS(x3)	mongo:7	interno	Replica Set Shard 1 (s1-1, s1-2, s1-3)
Shard2RS(x3)	mongo:7	interno	Replica Set Shard 2 (s2-1, s2-2, s2-3)
configRS(3x)	mongo:7	interno	Config Servers (config-1, 2, 3)
Postgres	postgres:16-alpine	5432 interno	App DB + Keycloak DB

3) DESCRIPCIÓN DE MICROSERVICIOS Y RESPONSABILIDADES

3.1) NGINX – LOAD BALANCER

Tecnología	NGINX (Alpine Docker)
Responsabilidad	Punto único de entrada al sistema. Redirige tráfico a las réplicas de la API Go mediante round-robin. Gestiona proxy_pass /search/ → :8080 para el Search-service. Redirige autenticación a Keycloak.
Configuración	<pre>upstream api { server api-replica-1:8000; server api-replica-2:8000; } proxy_pass http://api; proxy_set_header Host \$host;</pre>
Escalabilidad	Agregar más réplicas de API al upstream sin downtime.

3.2) API GO – R1 Y R2

Tecnología	Go 1.22+ (Fiber o net/http), compilado en contenedor Alpine
Responsabilidad	Contiene la lógica de negocio principal. Expone endpoints REST. Valida tokens JWT a través del Middleware. Interactúa con el Repository para operaciones CRUD. Envía requests al servicio Resquest para flujos externos.
Configuración y Réplicas	• Dos instancias activas tras NGINX para alta disponibilidad y balanceo de carga. • DB_DRIVER=postgres mongo, JWT_SECRET, KEYCLOAK_URL, REDIS_URL
Escalabilidad	Escalar es tan simple como cambiar replicas: 2 a replicas: N en docker-compose. Nginx ya está configurado como load balancer con round-robin entre las réplicas

3.3) KEYCLOAK – AUTH + JWT

Tecnología	Keycloak (imagen oficial), base de datos propia en PostgreSQL
Responsabilidad	Gestión de identidades: registro, login, refresh token. Emisión de tokens JWT firmados (RS256 / HS256). Gestión de roles y permisos (RBAC). Integración con la tabla de usuarios del App DB en PostgreSQL.
Configuración	El realm restaurant-system se importa automáticamente al arrancar via --import-realm. El JSON define: • Roles de realm: admin y client

	• Usuario inicial admin con password 123 y rol admin asignado La config manual adicional en la consola de Keycloak: 1. Client account → activar Direct Access Grants (permite login con username/password directo) 2. Crear Client Scope realm-roles (tipo Default, OpenID Connect) con un mapper tipo User Realm Role para que el JWT incluya los roles del usuario en el token 3. Asignar el scope realm-roles como default al client account Esto garantiza que el JWT que devuelve Keycloak incluye el campo roles que el middleware de la API puede leer para autorización.
Integración	La API valida tokens mediante la JWKS URI de Keycloak. El Middleware extrae roles del claim 'realm_access.roles'.

3.4) MIDDLEWARE – JWT + ROLES

Tecnología	Capa de middleware en Go (integrada en la API)
Responsabilidad	Intercepta cada request entrante. Valida la firma y expiración del JWT. Extrae roles del token y verifica permisos por ruta/método HTTP. Autenticación de roles por transacción (granularidad por endpoint).
Flujo	Request → extraer Bearer token → validar con clave pública Keycloak → extraer claims → verificar rol requerido → permitir/denegar acceso

3.5) REPOSITORY – CAPA DE ABSTRACCIÓN DE DATOS

Tecnología	Go — patrón Repository con interfaces
Responsabilidad	Abstrae el acceso a la base de datos. Lee la variable DB_DRIVER en tiempo de inicio para instanciar el driver correcto (PostgreSQL o MongoDB). Expone métodos CRUD independientes del motor subyacente.
Patrón	Se usa Repository porque el patrón trabaja por bloques intercambiables. La API habla únicamente con la interfaz y no sabe ni le importa qué hay detrás. La ventaja es que si se quiere agregar Oracle, CosmosDB, o cualquier otra

	DB, es solo crear un nuevo bloque. <pre>graph TD; HG[Handler Go] --> RI[Repository Interface
create, FindByID, Update, Delete...]; RI -- "factory.go
DB_DRIVE = DB" --> PR[postgresRepo]; RI -- "factory.go
DB_DRIVE = DB" --> MR[MongoRepo]; PR --> PG[Postgres]; MR --> MO[Mongo];</pre> Con DAO hubiera sido lo opuesto. Como DAO está acoplado directamente a la tecnología de acceso a datos (en nuestro caso GORM), cambiar de DB implica refactorizar cada DAO, y por consecuencia todo el código que los llama.
Switch de DB	El swap de DB se reduce a cambiar DB_DRIVER en .env. Factory.go inyecta la implementación correcta sin tocar un solo handler.

3.6) MONGO-ROUTER

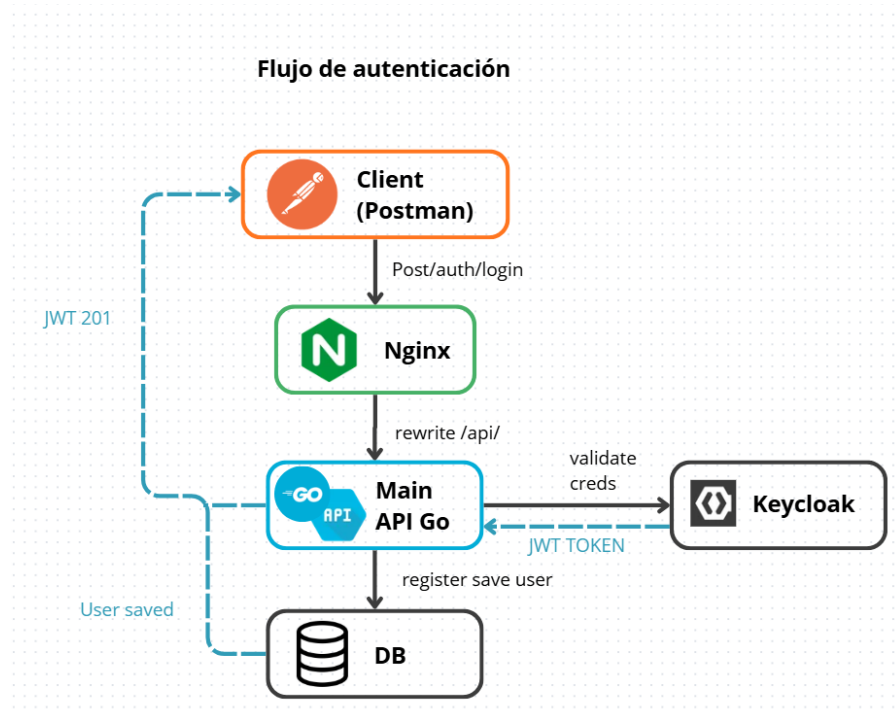
Tecnología	MongoDB mongos (enrutador de cluster sharded)
Responsabilidad	Punto de entrada al cluster MongoDB sharded. Enruta queries al shard correcto basándose en la shard key (id hashed). Transparente para la aplicación.
Colecciones sharded	product (hashed _id), reservation (hashed _id)
Escalabilidad	Distribución automática de datos entre shards. Alta disponibilidad gracias a Replica Sets en cada shard.

3.7) SEARCH-SERVICE

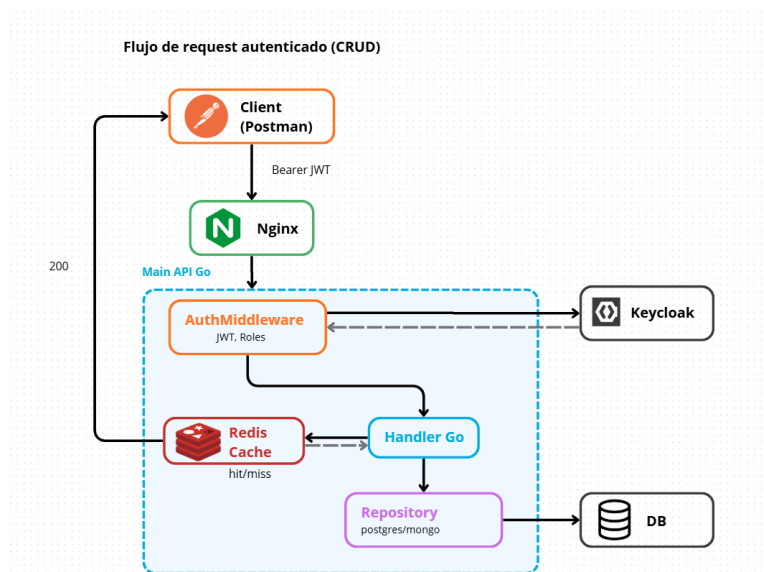
Tecnología	Go (servicio independiente), puerto 8080
Responsabilidad	Servicio dedicado a búsquedas full-text. Recibe queries del usuario vía NGINX proxy_pass /search/ → :8080. Consulta Elasticsearch y retorna resultados en formato JSON (hits).

4) FLUJO DE DATOS Y LOADBALANCES

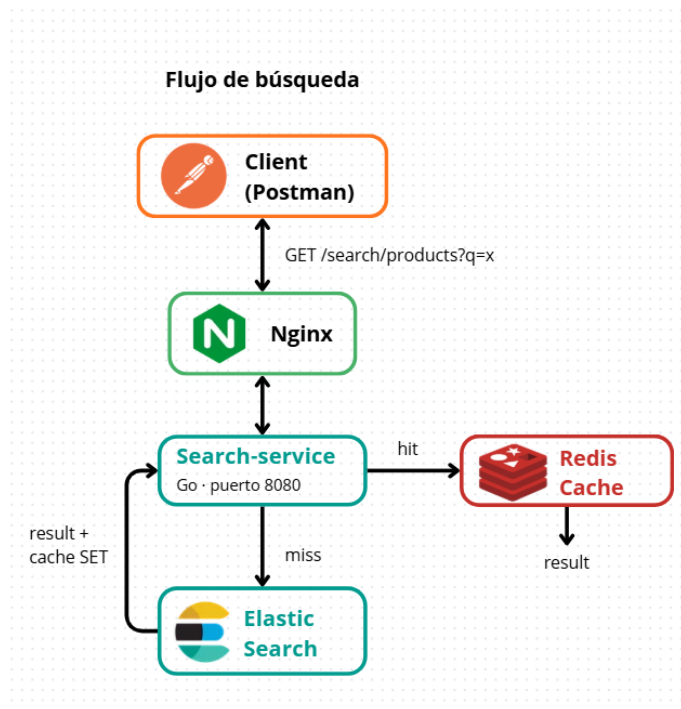
4.1 FLUJO DE AUTENTICACIÓN



4.2 FLUJO DE REQUEST AUTENTICADO (CRUD)



4.3 FLUJO DE BÚSQUEDA



5) PERSISTENCIA Y ESTRATEGIA DE BD

El sistema soporta dos motores de base de datos intercambiables. La variable de entorno `DB_DRIVER` controla cuál implementación del Repository se instancia al arrancar la API:

DB_DRIVER	Casos de uso
postgres	Datos relacionales, transacciones ACID, reportes
mongo	Datos documentales, alta escritura, colecciones grandes

6) CACHÉ Y BÚSQUEDA

6.1) REDIS

Se implementó de forma que antes de ir a la DB, el handler consulta Redis. Si hay HIT devuelve directo; si hay MISS consulta la DB, guarda en Redis y responde. En writes (CREATE/UPDATE/DELETE) se invalida la key correspondiente para evitar datos stale.

TTLs definidos en `cache.go`:

- `TTLRestaurants / TTLMenus` → 24h (cambian poco)
- `TTLOrders` → 5 min (cambian frecuentemente)
- `TTLSearch` → 5-15 min (balance entre frescura y performance)

6.2) ELASTICSEARCH

El índice `products` en ES contiene una copia de todos los `MenuItems` activos. No se consulta la DB en búsqueda — ES es la fuente para eso.

La sincronización es event-driven desde los handlers:

- CreateMenuItem → go indexInSearch() → ES indexa
- UpdateMenuItem → go indexInSearch() → ES actualiza
- DeleteMenuItem → go deleteFromSearch() → ES borra

El search-service es el único que habla con ES.

7) CI/CD — GITHUB ACTIONS

El pipeline se divide en dos jobs encadenados:

Job 1 — Test: corre en cada push o PR a main. Ejecuta las pruebas unitarias de internal/handlers/ y valida que la cobertura sea mínima 90%. Si no pasa, el pipeline falla y nunca llega al build. Esto garantiza que ningún código roto llega a producción.

Job 2 — Build & Push: solo corre si el Job 1 pasó y es un push directo a main (no PRs). Construye dos imágenes Docker y las publica en GitHub Container Registry (ghcr.io):

- restaurant-api — la API principal
- restaurant-search — el search service

Cada imagen se taguea con el SHA del commit (sha-xxxxxxx) para trazabilidad exacta, más un tag latest para facilidad de uso. Se usa Docker layer caching via GitHub Actions cache para acelerar builds repetidos.

8) ESCALABILIDAD – KUBERNETES

Los archivos YAML definen cómo correría el proyecto en un cluster real (AWS, GCP, etc.):

- **api-deployment.yaml** — define un Deployment con 3 réplicas de la API. Kubernetes se encarga de mantenerlas vivas — si una cae, la levanta automáticamente. La imagen viene directo de lo que GitHub Actions publicó en ghcr.io.
- **api-service.yaml** — define un Service tipo LoadBalancer. En un cloud provider esto provisiona automáticamente un balanceador de carga real que distribuye tráfico entre las 3 réplicas. El mundo ve el puerto 80, internamente llega al 8080 de cada réplica.